AI-POWERED ANSWER QUALITY ASSESSMENT AND VERIFICATION SYSTEM

A multi-model, evidence-based approach to evaluating AI-generated responses

Project focus	Answer evaluation, factual verification, hallucination detection, and model comparison
Core technologies	Python, Streamlit, LLMs, embeddings, and a vector database
Research component	Compare selected models using the same test dataset

1. Project Overview

We are building an AI-based system that evaluates answers generated by chatbots and Large Language Models (LLMs). It checks whether an answer is relevant, factually correct, complete, clear, and supported by reliable information. For example, if a chatbot gives an incorrect explanation of Retrieval-Augmented Generation (RAG), the system can compare its claims with reference documents, identify possible errors, and produce a quality score with suggestions for improvement.

2. Proposed AI Agents

1. Relevance Evaluator

Checks whether the generated answer directly addresses the user's question.

Output: Relevance score, explanation, and off-topic content.

2. Factual Accuracy Evaluator

Compares factual statements with reference documents or verified sources to identify possible errors.

Output: Accuracy score, incorrect claims, and supporting evidence.

3. Hallucination and Grounding Detector

Flags claims that are not supported by retrieved documents and highlights evidence gaps.

Output: Flagged claims, evidence references, and risk level.

4. Completeness and Clarity Evaluator

Checks coverage of important points, clarity, and unnecessary repetition.

Output: Completeness and clarity scores, plus missing points.

5. Bias and Safety Checker

Screens for potentially biased, inappropriate, or unsafe responses. Results are signals, not perfect judgments.

Output: Flagged text, issue category, and explanation.

6. Final Verdict Agent

Combines evaluation results, calculates the overall score, and assigns a verdict.

Output: Overall score, Pass / Needs Improvement / Fail, and recommendations.

3. AI Models We Can Use

A. Language models (LLMs)

• GPT models (OpenAI): useful for structured evaluation, reasoning, and feedback; API use may involve cost.
• Gemini models (Google): can analyze answers and generate structured evaluation reports.
• Llama (Meta): open-weight model family suitable for local experiments, depending on the chosen model and hardware.
• Mistral models: another option for answer analysis and hosted or local deployments.
• Hugging Face models, including FLAN-T5 and other text models: useful for experiments with generation and classification.

B. Embedding models

Model	Possible use
all-MiniLM-L6-v2	Lightweight semantic search
BAAI/bge-small-en-v1.5	Document retrieval

Model	Possible use
intfloat/e5-base-v2	Semantic similarity and retrieval
OpenAI embedding models	Text embeddings through an API

C. Additional evaluation models and techniques

Model / technique	Purpose
BERT / RoBERTa	Text classification and similarity-related tasks
NLI models	Check whether evidence supports or contradicts a claim
Cross-encoder rerankers	Reorder retrieved passages by relevance
BART-based MNLI models	Experimental entailment and claim-verification tasks
ROUGE / BLEU	Reference-text overlap metrics; not enough alone to judge factual correctness
BERTScore	Compare semantic similarity between generated and reference answers

4. RAG Workflow

1. Upload reference documents (PDF, TXT, DOCX, or a dataset).
2. Extract the text and split it into smaller chunks.
3. Generate embeddings for the chunks.
4. Store embeddings in a vector database.
5. Retrieve relevant passages for the submitted question and answer.
6. Provide retrieved evidence to the evaluation agents.
7. Generate scores, flagged claims, and a final report.

Input documents
Extract text and split into chunks
Embedding model → vector database
Retrieve relevant evidence
Evaluation agents inspect the answer
Score, verdict, and report

5. Project Modules

Input Management Module: Accept questions, chatbot answers, reference answers, and PDF uploads.

Knowledge Base Module: Extract documents, chunk text, create embeddings, and retrieve evidence.

AI Evaluation Module: Run relevance, accuracy, completeness, clarity, and hallucination checks.

Model Comparison Module: Evaluate the same answer with two or more selected models and compare results.

Scoring and Verdict Module: Calculate the final score and assign a verdict.

Batch Processing Module: Evaluate multiple answers through CSV or dataset uploads.

Dashboard and Reporting Module: Display scores, graphs, error categories, and downloadable reports.

6. Suggested Technology Stack

Component	Suggested option
Programming language	Python
User interface	Streamlit
Backend (optional)	FastAPI
Language model	One API-based or locally hosted model
Embedding model	all-MiniLM-L6-v2
Vector database	ChromaDB or FAISS
AI framework (optional)	LangChain or LlamaIndex
Data processing	Pandas
Charts	Plotly or Matplotlib

Component	Suggested option
PDF reports	ReportLab

7. Overall Scoring Formula

An initial weighted formula can be used and adjusted after testing:

$$\text{Score} = 0.25R + 0.35A + 0.20C + 0.20G$$

R = relevance, A = accuracy, C = completeness, and G = groundedness. Each component is scored out of 100. Accuracy receives the highest weight because a relevant and complete answer can still contain incorrect information.

Overall score	Verdict
80–100	Pass
60–79	Needs Improvement
Below 60	Fail

These thresholds are proposed starting points, not validated performance standards. Serious factual errors should also be flagged separately so that a high average score does not hide a critical mistake.

8. What Makes This Project Different?

• Model comparison: evaluate the same response using multiple models and compare their scores.

• Evidence-based error detection using reference documents.

• Unsupported-claim identification with source references.

• Automatic feedback and suggestions for improving answers.

• Batch evaluation and graphical reports.

• Optional tracking of processing time and API cost.

9. Recommended Implementation Plan

Start with a small working version using Python, Streamlit, one language model, an embedding model, and ChromaDB or FAISS. Add document upload and evidence retrieval first, then implement relevance, accuracy, completeness, and groundedness scoring. After the baseline works, add a second model for comparison. Test the system on a small dataset with manually checked reference answers, and compare evaluation agreement, factual error detection, processing time, and cost. This keeps the project manageable while providing a clear research component.

10. Conclusion

The AI-Powered Answer Quality Assessment and Verification System evaluates AI-generated responses using multiple evaluation agents, language models, embedding models, and a RAG-based knowledge base. It checks relevance, factual accuracy, completeness, clarity, and grounding, while highlighting unsupported claims and possible safety concerns. Its model-comparison feature allows different models to be tested on the same dataset. By combining evidence retrieval, structured scoring, batch processing, and report generation, the project aims to improve the reliability and transparency of chatbot responses. The proposed system can be developed incrementally, beginning with a simple prototype and expanding as results are evaluated.